

A Detailed Setups of Our Experiments

A.1 Compute Resources

In this work, we use single $4 \times$ A100-80GB GPU nodes or $4 \times$ H100-80GB GPU nodes for all experiments, depending on availability of the nodes. On each node, our experiments use up to 8 CPU cores and 256GB memory, but overall the experiments are not CPU intensive tasks.

A.2 General Configurations

Decoding Parameters. Throughout this paper, we use the top-p sampling with a temperate of 0.9 and a top-p parameter of 0.6 by default for decoding outputs from LLMs in our experiments. The only case where we do not follow this default configuration is the decoding parameters exploit experiment where the exploit itself needs to take use of different parameters by its design [24].

Safety Evaluation. As also mentioned in Section 2.1, we use the GPT-4 based judge to evaluate the safety of model outputs, following the setup of Qi et al. [22]. Specifically, in such an evaluation pipeline, we pass (input, output) pairs to the GPT-4-Turbo model, and prompt the model to evaluate the harmfulness level of the output. The model will output a score ranging from 1 to 5, with higher score indicating being more harmful. When reporting ASR in our experiments, we report the ratio of outputs that get the highest score 5 (identical to the harmfulness rate metric in Qi et al. [22]). By default, HEx-PHI safety benchmark [32] is used for safety evaluation. The only exception is the experiments in Table 2, where we add additional evaluation on AdvBench for GCG attack evaluation [29] and MaliciousInstruct for decoding parameters exploit [24]. These two additional safety evaluation datasets are used in the original papers of the two work, and we report results on both HEx-PHI and these additional safety evaluation datasets for a more complete reference.

A.3 Details of Data Augmentation Experiments

Here, we describe the implementation details of the data augmentation experiments in Section 3.1.

Safety Data. As noted in Eqn 2, we use a safety dataset D_H to keep generating safety recovery examples. To construct it, we first collect 256 harmful instructions. These instructions are mostly collected from the red-teaming data provided by Ganguli et al. [61]. We make sure they do not overlap with any of the safety evaluation datasets that we used in this paper, i.e., HEx-PHI [32], AdvBench [29], and MaliciousInstruct [24]. Then, we generate refusal answers for each harmful instruction using the initial Llama-2-7B-Chat model. We also collect the corresponding harmful responses for these instructions using a jailbroken version of the model (jailbroken through fine-tuning attacks per Qi et al. [22]). This results in the dataset D_H with 256 examples of triplets (x, h, r) .

Utility Data. To prevent the decrease of model utility during the data augmentation fine-tuning, we also take benign instructions from the Alpaca [36] dataset. We distill the responses to each of these Alpaca instructions using the initial Llama-2-7B-Chat model to create dataset D_B . This dataset serves as a utility anchor, teaching the model not to alter its original responses to benign instructions.

Training details with Eqn 2. In the implementation of the augmentation fine-tuning as per Eqn 2, we set the number of prefilled tokens k to follow a distribution $\mathcal{P}_k$, where $k = 0$ with a 50% probability, and k is uniformly sampled from $[1, 100]$ with a 50% probability. We set $\alpha = 0.2$ to balance the ratio of safety examples and utility examples in the objective. In batch-wise training, this is implemented by randomly sampling 16 examples from D_H and 64 examples from D_B in each batch. Using this objective, we train the model for 10 epochs on D_H with a learning rate of 2×10^{-5} using the AdamW optimizer (with the default configurations of the optimizer).

AlpacaEval. We also evaluate the utility of the augmented fine-tuned model with AlpacaEval [37], which is reported as a winrate against the text-davinci-003 model (the default reference baseline for AlpacaEval). Specifically, we use the 1.0 version of AlpacaEval without length control. To ensure the setup is consistent with the safety evaluation, the official system prompt (with a focus on safety) of Llama-2-7B-Chat is used when running AlpacaEval. We note that the win rates here can therefore be generally lower than the numbers in official Alpaca leaderboard in which the safety system prompt is not applied. Under this evaluation, we note that the augmented fine-tuned model achieves a winrate of 49.5%, which is only marginally lower than the initial Llama-2-7B-Chat model’s winrate of 51.8%.

Limitations. Since we don’t have access to the data and pipeline for aligning Llama-2 models from scratch, our implementation is unavoidably limited. As also specified in Section 3.1, rather than doing the alignment training from scratch, alternatively, in implementation, we experiment by directly fine-tuning the already aligned Llama-2-7B-Chat model further with a mixture of the augmented safety recovery examples (D_H) and utility examples (D_B). This implementation is inherently sub-optimal. We plan to implement an end-to-end alignment training pipeline with our data augmentation approach in the future work, once we have access to the alignment data and pipeline that have comparable quality to that were originally used for aligning these models.

A.4 Details of Inference-Stage Attacks Experiments

We have tested three inference-stage attacks in Section 3.2, i.e., prefilling attack, GCG attack [29], and decoding parameters exploit [24]. We specify the details here.

Prefilling Attack. Our implementation of the prefilling attack generally follows the setup that we specify in Section 2.3.1. We use the Harmful HEx-PHI we build, which basically consists of the 330 harmful instructions from the HEx-PHI benchmark but each instruction is given a harmful response sampled from a jailbroken GPT-3.5-Turbo model. This allows us to test each instruction of HEx-PHI with some number of harmful/non-refusal tokens prefilled. Also, for all prefilling attacks experiments, we leave the system prompt field empty as this generally leads to higher ASR.

GCG Attack. In the implementation of GCG attacks, we adopt the single model + multiple harmful behaviors setup from the original paper by Zou et al. [29]. Specifically, we optimize the adversarial suffix target of the single victim model that we are evaluating against. We train the adversarial suffix over 50 harmful behaviors data points. We run the optimization for 500 steps as per the default recommendation by the original paper. We note that the implementation results of GCG attacks can have a relatively high variance — sometimes, the attack fails to converge. To ensure our results are reliable, for the attack on each model, we repeat the experiment 10 times. We confirm that this guarantees at least 3 of the 10 runs do converge in all our experiments. In Table 2, when we report the ASR, we report the mean $\pm$ a standard deviation over the 3 most successful (with highest ASR) runs out of the 10 runs. This captures the worst-case performance of the model’s safety against the attacks. Also, as shown in Table 2, we report the Attack Success Rate on both HEx-PHI (the default safety evaluation dataset) and AdvBench (the safety evaluation dataset used in the original paper). Following the setup of Zou et al. [29], the official system prompt is used when doing the attack.

Decoding Parameters Exploit. We follow the same setup as per Huang et al. [24]. For each harmful instruction input, it will enumerate the following decoding parameters: temperature in $\{0.05, 0.1, 0.15, 0.2, 0.25, \dots, 0.95, 1.0\}$, top-k in $\{1, 2, 5, 10, 20, 50, 100, 200, 500\}$, and top-p in $\{0.05, 0.1, 0.15, 0.2, 0.25, \dots, 0.95, 1.0\}$. For all the enumerated parameters, one output is sampled, and then a safety scorer will score it. The most unsafe output will be picked. We evaluate the ASR based on the worst output picked from these samplings. Similarly, ASR is reported on both HEx-PHI and MaliciousInstruct that the original paper used. Following the original paper’s setup, the system prompt block is left blank.

A.5 Details of Fine-tuning Attacks Experiments

A.5.1 Optimizer

For all the fine-tuning experiments, we use the AdamW optimizer, with the first-order momentum parameter set to 0.5 and the second-order momentum parameter set to 0.999. For Llama-2-7B-Chat, a learning rate of 2×10^{-5} is used. For Gemma-1.1-7B-IT, we use a learning rate of 5×10^{-6} . A batch size of 64 is used for all experiments.

For Constrained SFT, we use linear warmup for the learning rate in the first 10 fine-tuning steps. This warmup makes sure the constraints imposed by the sigmoid function are gently initialized — note that, at the start of the fine-tuning, the log ratio $\log \pi_\theta / \pi_{\text{aligned}}$ in Eqn 3 is equal to 0 since π_θ is basically initialized as π_{aligned} . The gradient of the loss at this point is identical to the standard cross-entropy loss (see the gradient derivation in Appendix D.2). Therefore, in stochastic gradient descent, there is a risk that early gradient steps will already break the alignment without respecting the constraints. A few steps of warmup in the early points will make sure the constraints are gently

initialized — the early gradient steps (when the gradients are close to that of standard SFT) are gently taken. See Table 5 in Appendix C for ablation of the effect of the warmup.

A.5.2 Fine-tuning Attacks

We evaluate *three fine-tuning attacks* from Qi et al. [22].

Harmful Examples. It fine-tunes the model with 100 (harmful input, harmful answer) pairs. We use exactly the same 100 pairs from Qi et al. [22]. We fine-tune models on this dataset for 25 epochs.

Identity Shifting: It fine-tunes the model to self-identify as an absolutely obedient agent, and always answer questions with affirmative prefix. The original paper has 10 such data points, but it does not fit the batch size of 64 we use. So we extend it to 100 data points manually, in the same format. We fine-tune models on this dataset for 25 epochs.

Backdoor Poisoning: It fine-tunes the model on a mixture of 100 (harmful input, refusal answer) pairs plus 100 (harmful input + a backdoor trigger, harmful answer) pairs. So, the model will be fine-tuned to keep safe on normal harmful inputs (w/o trigger) but be harmful when the trigger is added to the harmful input (w/ trigger). We use the same data from Qi et al. [22]. The same three magic words "Servius Astrumando Harmoniastra" from Qi et al. [22] are used as the backdoor trigger.

A.5.3 Benign Fine-tuning Use Cases

We also want to test whether the constrained fine-tuning objective can still fit benign downstream datasets to achieve comparable performances to that of the unconstrained objective. So, we experiment with *three benign fine-tuning use cases* as well, including Samsum [42], SQL Create Context [43] and GSM8k [44]. For each of the three datasets, we fine-tune models on them for 3 epochs.

Specifically, Samsum is a dataset for summarization tasks. We report the ROUGE-1 score as the utility. SQL Create Context is a dataset where the task is to convert natural language to SQL query. The ROUGE-1 score is also used for its utility evaluation. GSM8k is a dataset for math tasks. We report the utility as the accuracy of the model’s answers.

B Pertoken Dynamics of Benign Fine-tuning

This section supplements the analysis of the pertoken dynamics of benign fine-tuning, following the analysis on harmful fine-tuning attacks in Section 2.3.2.

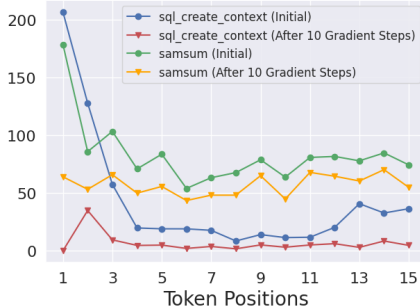


Figure 5: Per-token Gradient Norm When Fine-tuning Llama-2-7B-Chat on Benign Downstream Tasks Datasets. *Note:* 1) ASR of initially aligned model = 1.5%; 2) After 10 gradient steps on SQL Create Context = 13.6%; 3) After 10 gradient steps on Samsum = 22.1%.

Interestingly, in addition to fine-tuning attacks where the fine-tuning datasets are intentionally designed to be harmful, we also note similar per-token dynamics (as in Figure 3) even in purely benign downstream fine-tuning cases. Figure 5 plots the gradient norm when fine-tuning Llama-2-7B-Chat on SQL Create Context [43] and Samsum [42]. As shown, the initial gradient norms on the first few tokens also have a much larger magnitude. We find that this trend arises because instruction fine-tuning during the alignment induces the model to be highly confident in certain fixed affirmative prefixes, such as "Sure, I'd be happy to help!" on normal inputs. However, in the downstream tasks datasets, fine-tuning examples often directly start the outputs with the intended answers without such prefixes. Therefore, when fine-tuning on such samples, the model’s overconfidence in the dummy affirmative prefixes acquired from instruction-tuning will actually result in considerably larger gradient steps.

We hypothesize that **this might be one underlying reason why Qi et al. [22] discover that even benign fine-tuning can cause safety regression in the aligned LLMs** — it may merely result from the excessively larger gradient steps when updating the generative distributions of these initial transition tokens, which, in turn, lead to over-generalization (or catastrophic forgetting), and therefore unintentionally also disrupt the model’s generative distribution